

Active Inference LiveStream 056.0 ~ Neural coding, Predictive processing,

Livestream | Neural coding, Predictive processing, and Cognitive modeling. | 1:19:17

Hello and welcome everyone. It is November 2nd, 2023, and Blue and I are here in ACTIM live stream 56.0, kicking off the 56 series on a set of four papers.

Welcome to the Active Inference Institute. We are a participatory online institute that is communicating, learning, and practicing applied active inference. You can find us at links here.

This is a recorded and an archived live stream, so please provide feedback so we can improve our work.

All backgrounds and perspectives are welcome and will follow video etiquette for live streams.

Head over to activeinference.org if you want to learn more about the institute,

get involved in any activities or learning groups. Let's begin with a little bit of an introduction

and warm up, say hello, and then maybe just one thing that we found interesting about these papers

or that made us want to jump in for the 56 series. So I'm Daniel. I'm a researcher in California,

and I was very interested to learn about what the relationship is between different neural

inspired approaches to machine learning and active inference. Blue?

Blue?

Blue?

I mean, same, this kind of always like the neurobiologically inspired

machine learning is always kind of something that piques my interest and I'm still here, still

excited about it. And in this like set of papers, I was actually like,

particularly interested in like sparsity and how that is maybe something that's particularly

biological or not, or just kind of learning and diving deeper into sparsity as like a topic of consideration.

Cool. All right. Well, in this series and in this video, we're going to discuss four papers,

and we'll get to why we have four soon. Four papers are the Neural Coding Framework for Learning Generative Models, 2022.

Convolutional Neural Generative Coding Scaling Predictive Coding to Natural Images, 2022.

Spiking Neural Predictive Coding for Continually Learning from Data Streams, 2023.

And CogNGen, Building the Kernel for Hyperdimensional Predictive Processing Cognitive Architecture, 2022.

And as there are four papers, we're going to use acronyms for each paper and also go through them in this order.

And again, we'll return to why there's four and why this order. The four acronyms are NCF-22,

CGN-C-22, SNPC-23, PPCA-22. So the video is going to be an introduction. We're going to cover

the big questions and then basically jump immediately into the four papers. And this is just an introduction.

Feel free to write comments or if you're around in time to join for these upcoming discussions that

we're going to have with Author. So, Blue, what big questions are you excited about here?

So I think I already talked about sparsity, but I would like to know what other types of tasks that

are perhaps maybe non-visual could Neural Generative Coding be suited to? And then the concept of,

in the last paper, we'll get to it, of a memory echo as a prediction. I think,

is prediction just a memory echo? And then that leads into what is the difference between I predict, or I expect, or I hope, right? I predict that the Eagles might win the Super Bowl with maybe 50 to 60%. That's the probability, right? 50 to 60% probability that the Eagles win the Super Bowl, but I totally hope that the Eagles win the Super Bowl 100%. So where's the difference between hoping? And I can actually, I thought about it and it's like action, the role, you hope something happens, you hope you get into a good college, you predict that you'll go to college, you hope you get into a good college, you get good grades, do all the things, and then you can act based on that hope. But I actually can take no action to affect whether or not the Eagles win the Super Bowl. So action is not necessarily a key role there, but how is hope different from prediction and expectation? And how would you approach modeling something like that? It's just something that derailed me anyway. That's interesting. Yep. So my big questions first, just on the scope. How is it different now that we're in our first quadruple dot zero? We've done one paper at once, we've done two at once, side by side, four is double side by side, and we love the four. And we're going to get a little bit more. About the topics though, we are going to get very micro and zoomed in, and we're going to have some of the formalisms brought in, but there's a lot more in the paper. So how do these really mechanistic and or abstract micro level discussions of like neuron spiking help us get closer to real cognitive architectures and modeling? And then I think to kind of for, um, warn of the fourth paper, here's where we're going to see something like the cognitive architecture that includes not just perception and memory and all these things, but also the action component. So how does this connect to active inference? Like, is this driving down the low road coming from the high road? Is it another road? So where's the work that we're going to be looking at in this sequence of research related to Bayesian mechanics, free energy principle, some of these more first principles, intelligence conversations that we have. So let's talk about why there are four papers. So I hope this is okay. That from Aurobia, I share something that he wrote. He suggested the four papers and said, almost in that sort of order from the more base neural generative model coding, predictive coding framework, and CF-22, that's going to be the first paper we discuss to its convolutional generalization, CNGC 22, down to its spike level generalization SNPC 23, and back to its part in generalizing active inference to a robotic setting, PPCA 22, in a sort of progression slash series. So hopefully this lays out our approach to these four papers, which we were happy to accept this curriculum and challenge. The first paper that we're going to look at is going to set the kind of base canvas or overarching umbrella model, the NGC. Then there's going to be a convolutional and a spike level generalization or kind of additional motif. And then we'll get to this question of more integrated cognitive modeling in the fourth bit. Any overall thoughts just on the four papers, Blue, before we go to the first one? I mean, I'm curious to ask, like, what was the order that these like started in? And like, how, like, what, like, you know, I mean, sometimes you just finish everything at once, or sometimes like you start something that takes forever. So I'd like to know, like, what were the order that the projects were started in, as opposed to like, I mean, we'd have the publication dates and whatever. But I would like to know, like, did they start and go in this order,

just as like a narrative from the author? That's a great question. All right, first paper, Neural Coding Framework for Learning Generative Models, Aurobia and Kiffer 2022. So just let me know when to switch slides for this section, Blue. Thank you. Awesome. Okay, so we'll read the abstract. I'll go, I'll just, maybe I'll take a couple sentences and then pass it to you, Daniel, if you don't mind. Okay. Neural generative models can be used to learn complex probability distributions from data, to sample from them, and to produce probability density estimates. We propose a computational framework for developing neural generative models inspired by the theory of predictive processing in the brain. According to predictive processing theory, the neurons in the brain form a hierarchy in which neurons in one level form expectations about sensory inputs from another level.

Do you want to read the rest? These neurons update their local models based on differences between their expectations and the observed signals. In a similar way, artificial neurons in our generative models predict what neighboring neurons will do and adjust their parameters based on how well the predictions match reality. In this work, we show that the neural generative models learned within our framework perform well in practice across several benchmark datasets and metrics, and either remain competitive with or significantly outperform other generative models with similar functionality such as variational auto-encoder.

Cool. Thanks. So the roadmap, this is a nature paper and it's dense. So there are like 18 different formalisms in this paper and like many, many different models and sets of experiments. So the roadmap is here,

and we are not going to go that deep into this because we're doing four papers and you know, hopefully under 90 minutes.

And so we're going to go deep into the maths here, but hopefully we can get more heavy into it in the dot one. So pretty much introduction results, discussion and methods, but really what happens here is the results section kind of gives a lot more introductory information about like the back propagation and what learning looks like in the brain and the neural coding framework and how this kind of, you know, forms a bridge here and then training the model and how the model updates everything and then what can the model actually do. So we're going to kind of get into a little bit about the model and what the model can actually do. And then Daniel, I think we'll delve deeper into the next couple of papers into the model, and then finally tie it back into active inference in the final paper. So just that's an overall roadmap, and then the roadmap for this paper too. So go ahead and flip it to the next one.

All right. So in this paper, the author states that in neurobiological systems, neurons make use of the information immediately available to them in both time and space and do not wait on distant regions in order to adjust their synapses with global information provided through neurotransmitters such as dopamine. Also later in the paper, he says, while the question as to how credit assignment is exactly implemented in the brain is an open one, it would prove useful to machine learning, computational neuroscience, and cognitive science to have a framework that demonstrates how a neural system can learn something as complex as a generative model without backpropagation using mechanisms and rules that are brain inspired. So that's kind of what they set out to do in this

paper. So let's go ahead and flip it.

So here, this is figure one, and this is like figure 1A looking at a backpropagation model.

And so this is from the legend in the figure. He says credit assignment in backpropagation or in backprop. I won't keep elongating that credit assignment in backprop requires a strict global feedback pathway, which requires the completion of the forward pass that carries information upstream, right to left. This feedback pathway carries the error message E_0 at the output layer back along left to right, the same synapses used in the forward pass to update downstream neurons Z_1 and Z_2 . Backprop differs from brain-like learning because, so this is within the paper, not the legend, the synapses that make up the forward information pathway need to be directly used in reverse to communicate teaching signals, which is the, this is known as the weight transport problem. And so you can see that here. So here it's going from left to right, like the output layer is here at E_0 , or the error message is here at E_0 . And then the backpropagation happens is here at this red arrow. Thank you, Daniel. As I'm pointing to the screen, you're following my finger along. Okay. Neurons need to know about and

communicate their own activation functions first derivative. And that is shown here in the little deltas. So neurons must wait for the neurons ahead of them to percolate their own error signals backwards before they know when or how to adjust their own synapses. And this is known as the update locking problem. And then also there is a distinct form of information propagation for error feedback, one that starts from the system's output and works its way back to the input layer. So that's shown here in figure one, which does not influence neural activity. So in other words, the global feedback pathway problem backprop creates signals that only affect weights, but do not at least directly affect or improve the network, the network's representation of the environment. So it seems like a lot of extra steps. And then the error signals have a one to one correspondence with neurons. So we can flip to the next side. So in this paper, the first four of those points are addressed by neural generative coding. So everything except the final point, the one to one correspondence between the error signal and the neuron. So this is the neural generative coding model. So it says the proposed NGC model sidesteps this neurobiologically implausible requirement by learning with short local error transmission pathways pathways that are made possible through recurrent error synapses and stateful neural activities. Credit assignment under neural generative coding operates with local mismatch signals E_1 and E_2 that readily communicate this

information to their respective layers Z_1 and Z_2 . So here the black arrows are forward propagation while the red arrows

are backwards transmission. Solid lines indicate that a signal is transformed along a synapse while dashed lines

indicate direct copying of information. So it's like the identity function, no transformation of the signal.

And then the delta shows the communication of the neurons first derivative. The triangle represents the computed change to the synapse of the forward pass that will use the nearby error signal. And then the dot is the multiplication of incoming signals, which you see those all like we don't see the dot here, but it's in the

the backprop model. So in the previous one. Okay, so that's the model. That's all we're going to talk about for this so far so far. So we're going to see now like what are the different models that they compared against. So I'm just going to list them right here. So they used backprop models, the regular regularized autoencoder. And these are the abbreviations that are used here. The Gaussian variational autoencoder, the constant various constant variance Gaussian variational autoencoder, the adversarial autoencoder, and then the deep sparse rectifier network. And that's purely discriminatively trained. So I just want to point that out. And it'll be important as we get later, just the last one. And then the neural generative coding models. So this is actually the first one is from the like super seminal Rao and Ballard 1999 paper. And then also a super seminal paper by Friston in 2008. And then the later two models, which we'll talk about more in the dot one are models that are developed by the authors of this paper. So these GNCN models are both developed by the author. And then the Gaussian mixture model is like just another classic benchmark model that's not backpropagation or the generative coding. It's just, yeah, a mixture of a Gaussian distribution. So, okay, next.

So what can these models do? So generative neural coding can learn viable auto associative generative models. And let's check out some of the results that they got. All right, so here we are table one. So I gave you all the models so that you can kind of see what's over here. So the only ones that are neural generative coding are the GNCN ones. So the last four, and then the first two, the first one's from the Rao paper that says Rao. And then the second one with the sigma is the Friston. So we see here the regularized autoencoder, auto associative network least equipped to serve as a data synthesizer model. So the other GVAE ones. And then while the variational autoencoder models yield worse reconstruction than the regularized autoencoder, they obtain much better log likelihoods, especially compared to the Gaussian mixed model baseline, indicating that they are strong data samplers. And then the neural generative coding models, the ones developed by the authors here, obtain competitive log likelihood with the variational autoencoder models. And with the GNCN PDH yields the best log likelihood of all of them for all four datasets. And so the datasets that they're talking about here, we're going to see some pictures of them in a little bit, but they're like the number datasets, the MNIST. So not all of them are numbers. So MNIST is like the classic number dataset. And then the FMNIST is like pieces of clothes. And then the Caltech one is like, I don't know, some strange shapes. So let's flip to the next one. And maybe it'll show us I think it does actually. Oh, no, let's flip to back to figure four so we can look at the datasets or to figure the next one. Yeah, and then we'll flip back. Yeah, let's let's see the datasets. There you go. Perfect. So sorry, I should have reordered these. But you can see I'm trying to describe these samples. So this one is like, some like handwriting. I'm not sure is that language Daniel, the KMNIST? Do you know?

Daniel K. No.

Daniel K. No.

It's you know, it's not any language. I was like, is it like, I don't know.

Is it Chinese Arabic? I could I can't read it. So I don't know. But you can see here,

the MNIST is numbers, the FMNIST is clothes, the count and like the rest are just like symbolic representations. So just so you guys know, this is what they're looking at. This is what they're trying to generate. So then let's go back to figure four.

Oh, there we go. Okay, so figure four. This is the error, the BCE loss measurements. So lower is better for pattern reconstruction on data subsets. So this is measured in terms of the percentage of the original database samples. So here you can see that the BCE loss, which is like, we want less, less is better for all of the different models here shown. So the RAE is blue. The variational is light green, the GAN, AE, that's like another auto encoder that is dark green. And then the three, the pink, purple and orange are neural generative coding models from first in and then the two shown here. So like they're not always, they don't always show every model in every figure. But you can see here that the error for the three, like the pink, pink, orange, pink, pink, orange is pretty low all the way across the board here. So I think that that's the important result. And then let's get we can flip on. So neural generative coding also yields strong downstream pattern classifiers. So let's see how the authors found that. In table two, in terms of test error, we can see them all here. So here's the different models. The DSRN is the discriminatively trained model. And then that's the auto encoder, the back, these are all the back propagation ones, the first four, and then the last four are the neural generative coding models. And with the final two being the ones used here, developed here by the authors. So we can see the data sets. So the top two of each, the top two pictures in each row are the data sets like from the original data. And then the bottom two are generated by the model. So we can see that this is how they are, like how they're generating the data samples. So here the the bottom two, even the very bottom one. Wait, yeah, the bottom three are competitive with the purely discriminatively trained model, the top one, which has the best output. And they outperform all of the generative models, even the DSRN in one case. So the bottom, the bottom model, bottom two models do really well in generating data. So good. And then figure six, I think shows kind of the same thing. So in figure six, a, this is a TSN plot of the data and the data clustering. So we provide qualitative evidence that the latent representations of neural generative coding, this is from the NCN PDH,

they appear to yield a stronger natural separation of the test data into class respective clusters compared with the back propagation based model, which was this was the best performing backprop model with respect to log likelihood and reconstruction error. So we can see that there's like a more distinct clustering here. And I think that that's, that's what the authors are saying is the qualitative evidence. Again, we emphasize that the neural generative coding model acquired representations without labeled information. So the class based relationships have emerged as a result of very sparse neural activities. This is why like I'm so interested in the sparsity here. This offers some promising evidence that the neural generative coding models representations offer benefits beyond the original density estimation task, allowing reuse of the same system for downstream tasks like categorization. So pretty neat. And then neural generative coding can conduct pattern completion. We'll click on the next one. So this, they didn't show the data here, they showed like a little bit in

the methods. But they said that another interesting ability attributed to auto-associative memory models, associative memory models, is their ability to complete partially corrupted or incomplete patterns. And they hypothesize that a neural generative coding model's ability to conduct pattern completion better than the auto-encoder models is due to its iterative inference processes. So I'd like to see future work that kind of probes at this a little bit.

Thank you, Blue. That's super helpful. Very nice. Any other comments on the paper?

No. Yeah. They replaced this time lock, weight transport bound, global feedback constraint setting with a biologically inspired, locally computable, etc., etc., architecture.

So that is kind of the base case that we're now going to dive into two specifications or elaborations of convolutional first, then spike level. And then in the fourth paper, it's going to come back to mainly blue with the overview of the cognitive architecture. Okay. Can you read the abstract for this one?

Sure. In this work, we develop convolutional neural generative coding, ConvNGC, a generalization of predictive coding to the case of convolution-deconvolution-based computation. Specifically, we correctly implement a flexible neurobiologically motivated algorithm that progressively refines latent state feature maps in order to dynamically form a more accurate internal representation slash reconstruction model of natural images. The performance of the resulting sensory processing system is evaluated on complex data sets such as the color minced, CIFAR-10, and street house view numbers.

We study the effectiveness of our brain-inspired model on the tasks of reconstruction and image denoising and find that it is competitive with convolutional auto-encoding systems trained by backpropagation of errors and outperforms them with respect to out-of-distribution reconstruction, including the full 90K CINIC-10 test set. Thank you. So this paper is somewhat similar and related to the previous paper, except as promised, it's going to generalize it into the convolutional setting. So there's also a lot of nice exposition in this paper, so it will be good to get some background too. Introduction, then the convolutional neural generative coding model, details on the theory and the practice of the experiments about what was tested and done, and they summarize contributions as follows. One, we propose a new neural perception model, CONV-NGC, which acquires robust representations of natural images in an unsupervised fashion. Two, to the best of our knowledge, this is the first work in the literature where visual inputs are processed using deep deconvolutional layers directly and naturally within a framework of predictive coding, significantly enhancing its representation power for vision-based tasks. And three, we demonstrate on several natural image datasets, that the proposed CONV-NGC is competitive with existing back-prop-based models of similar architectural designs on the tasks of image reconstruction and image denoising and outperforms them with respect to out-of-distribution prediction. So some of the first sentences of this paper that set the stage really well. The first sentence begins with back-propagation and highlighting that while it's done well, riding on the tide of increasing power of hardware and software, there are still some limitations associated with back-prop-based AI. The field still has a long way to go towards developing

artificial general intelligence. I think it'll be an interesting question. Is that what this field or who in this field means what by that? In order to increase task-level performance, the size of deep networks has increased greatly over the years up to hundreds of billions of synaptic parameters as seen in modern-day transformer networks. This scaling-is-all-you-need paradigm increases the number of parameters and the deployment of multiple motifs, like transformer architectures, attention motifs, and so on, as well as changing and curating datasets. And we are now getting up to models in the 2020s that have billions of parameters. However, this trend has started to raise concerns related to energy consumption and more. Energy consumption, limited flexibility and generalization, limited biological plausibility, and this results in some computational constraints and limitations, especially around parallelization, federation, learning, all these kinds of settings that we want to have effective algorithms for.

So, people are looking into different methods, biologically based and otherwise, that speak to different aspects of this. They write, some of the most promising pathways in addressing all those problems listed previously come from the emerging domain of research known as brain-inspired computation, which seeks to develop neural architectures and their respective credit assignment algorithms that leverage only local information motivated strongly by how learning is conducted in the brain.

The promise of brain-inspired computing brings with it synaptic adjustment mechanisms that are neurobiologically grounded, as well as neural computation and inference that is flexible, capable of providing a wide variety of operations, at biologically more faithful levels, facilitating massive algorithmic parallelization at scale, and adaption on analog and neuromorphic hardware.

So, it's an interesting discussion topics.

What aspects, what aspects or properties or processes or features or perspectives are we trying to copy about the brain?

And while we were preparing for this in some open sessions, someone told me about this spy kid's third brain,

and this kind of designing a brain, and what does it mean to really design cognitive architectures?

What does it mean to consider diverse intelligences?

How is this related to the kinds of large-scale scientific research projects that are going on in the human brain?

Any thoughts on that, Blue?

Okay.

At the end of the introduction, they again summarize their contributions that we looked at previously.

And before we jump in, this is from the appendix of the paper.

Here's some key symbol operator abbreviation definitions in Table 3.

A little bit of exposure to the symbols we're going to see.

Section 2. Convolutional Neural Generative Coding.

We start by describing our model instantiation of a convolutional neural generative coding, convmvgc, which is tasked with learning from streams of natural images in an unsupervised

fashion.

Typically, in a visual recognition task, input patterns belonging to different object classes arranged into batches are presented to a processing system at different time points for training. Here, they formalize the problem and give a bit more information on the notation.

Convngc is a generalization of the NGC computational framework from 41.

That was the NCF 22 paper that we just heard about.

And generalizing it to the case of natural image data.

So in those other data sets like MNIST and others, every pixel is just passed in of a relatively small image. So the whole image can be taken in as a matrix, like a binary matrix with black and white squares or a grayscale matrix. However, real visual data coming in, for example, off of a camera sensor is apt to have a much higher resolution. And so a biologically inspired design pattern that we find in the retina and other kinds of biological visual sensors is this idea of convolution, where a convolving function is used to blur and essentially get a coarse graining across a high resolution visual input. The underlying process of ConvNGc can be divided into three components. Local prediction and error unit map calculation, latent state map correction, and local synaptic adjustment. So this is drawing on the architecture and the motifs that were introduced previously. So this is drawing on the architecture and the model of the NGC model.

Here's the ConvNGc model. It consists of a set of L predictive layers, typically arranged hierarchically, though this is not a strict architectural requirement.

E.g. Figure 1 depicts a non-hierarchical NGC circuit. So that was Figure 1.

Here's Figure 2, which is, I guess, a more hierarchically structured ConvNGc with three layers.

Section 3 goes into more detail on this model. In this section, we provide concrete implementation details of the neural coding process described earlier, depicting how layer-wise state prediction, state correction, and synaptic parameter updating occurs specifically in the context of visual object reconstruction. The first two steps of the algorithm iteratively predict and correct the representations of the ConvNGc model for observed input values, natural images of the current dataset.

After T iterations, the final step entails adjusting model synaptic efficacies using simple Hebbian-like updates. So these are the three main steps. There's a two-stroke engine with prediction and correction, kind of like an expectation maximization. And then after a certain number of iterations T , there's the adjustment of synaptic efficacies. So here is from the appendix. This is the pseudocode of the algorithm.

Section 3.1. Inference, predicting and correcting neural states. Going through those steps. Layer-wise state map prediction and state map correction. Then 3.2. Training. Updating the model's parameters. Having described the model, essentially, they go to Section 4, the experiment.

We compare ConvNGc to powerful back-prop-based models such as the convolutional autoencoder, denoising convolutional autoencoder, as well as the fully connected form of neural generative coding, which under certain conditions recovers the unsupervised form of the model in 55, which is Salvatore et al. Hierarchical Predictive Coding Network.

Details are provided about the datasets used. And tables 1 and 2 provide some performance measures evaluated in terms of mean squared error, structural similarity index measures, and peak signal-to-noise ratios.

One can look to the tables to see where the different models do well or not.

Here are figures 4 and 3. Figure 3 shows an image randomly sampled from a dataset. Left, the same image corrupted with noise. So here's the original image, a 5 and some kind of object. And then a digitally corrupted version with a normally distributed noise term added. And then after processing, this is the denoised image. That's figure 3. Figure 4. Feature maps for the bottom three layers of a trained ConvNGC model. So this just shows a little bit of kind of the inner workings of the model. And surprisingly, with respect to the out-of-distribution experiments, OOD, ConvNGC outperforms all the baseline models. This desirable improvement is indicated by the higher trial-averaged out-of-distribution peak signal-to-noise ratio, as well as the structural similarity scores.

What is most impressive is that the ConvNGC model was trained on natural images in the housing numbers dataset that did not contain any of the objects found within either CIFAR-10 and CENIC-10, and was still able to reconstruct with top performance. This result offers a promising future direction worth exploring for ConvNGC and NGC predictive coding systems in general, their ability to generalize to unseen patterns that violate the assumption that they were generated by a distribution similar to that of a training data. So this is, I think, going to be an interesting discussion point. It's kind of like if you classified a visual detector based upon only a data set with a certain kind of edge or surface, then it's possible that the model wouldn't learn more general features of edges and surfaces. It might be able to use a back-propagation type updating and credit assignment paradigm so that something really, really high level and abstract, like whether it's a cat or a dog, could be tagged with something very micro, which is overfit to cats versus dogs, or even more locally, like that cat versus that dog data set. And so potentially constraining to these local connections only, making a kind of enforced sparsity means that the messages that get passed are not just more biologically plausible and fewer of them, but potentially they extract features more like the visual system does in terms of extracting generalized edges and features through lower level features that kind of percolate up into higher level features in a really methodical way, which is why we can like recognize a shape that we've never seen before. So in conclusion, here's the whole conclusion. Our results mark an important step towards crafting more robust general brain inspired neural architectures and learning processes capable of handling complex machine learning tasks.

So some questions for discussion are written here. And there are three very informative appendices in this paper. So for more detail, if you think this is cool, definitely read the paper, check out the appendices.

Onto the third paper. This is SNPC 23. Want to add anything on that, Blue?

Spiking neural predictive coding for continually learning from data streams.

Do you want to read the abstract?

Sure. For energy efficient computation in specialized neuromorphic hardware, we present spiking neural coding and instantiation of a family of artificial neural models grounded in the theory of

predictive coding. This model, the first of its kind, works by operating in a never-ending process of guess and check, where neurons predict the activity values of one another and then adjust their own activities to make better future predictions. The iterative, interactive nature of our system fits well into the continuous time formulation of sensory stream prediction and, as we show, the model structure yields a local synaptic update rule, which can be used to complement or as an alternative to online spike timing dependent plasticity. In this article, we experiment with an instantiation of our model consisting of leaky integrate and fire units. However, the framework within which our system is situated can naturally incorporate more complex neurons such as the Hodgkin-Huxley model.

Our experimental results in pattern recognition demonstrate the potential of the model when binary spike trains are the primary paradigm for inter-neuron communication. Notably, spiking neural coding is competitive in terms of classification performance and experiences less forgetting when learning from a task sequence, offering a more computationally economical, biologically motivated alternative to popular artificial neural networks.

Awesome. Thank you. Okay, here's the roadmap and the key piece of this paper. So they start with the introduction, then go to the methodology, define the problem really well, and introduce the spiking neural coding framework in order to implement this kind of continual error correction from the spike trains. Section 3 covers the experimental results, simulations, and limitations. Then there's conclusion and an appendix. They write, in this paper, we propose spiking neural coding, as well as a concrete model instantiation that is composed of leaky integrate and fire neurons and its learning process for adjusting synaptic weight strength incrementally. Although we only examined one kind of spiking neural models, one kind of spiking neural neural framework, our general framework can accommodate simulated neural models of varying complexity. So it's going to be a simple version of spiking with a binary spike train and a leaky integrate and fire neuron. However, there's even sub generalizations within this spiking paradigm. So just as a kind of reminder, we introduced the general local compute-only predictive coding architecture in the first paper. Second paper looked into the convolutional generalization. Both the first and the second paper, though, were trained in essentially batch mode. Training datasets were loaded in and they were trained in a sort of batch-based approach. Here, through this spiking mechanism, think about a trace through time and spikes happening along that timeline. The data are coming in not in a batch that the computer gets to process at the kind of pace or way it wants, but rather the data are coming in real time. So this generalization is much more fit to both the real-time like Internet of Things semantic spatial web setting, but also the biological perception setting. Here, there's a similar introduction. Instead of just harping on backprop though, artificial neural networks, ANNs are the models which are described in terms of being very large and powerful, but with challenges. And surprise, surprise, people are using brain-inspired computing to formalize and implement various kinds of neural computation and learning mechanisms currently known in cognitive neuroscience. And we only bring a few of these onto the slides, but I have been taking many sections of text and passing it through different image generators. So it's always fun to like see visual semantics of what sometimes these otherwise really wordy sections are about. So section 2, 2.1, problem definition, continually predicting sensory stream inputs. So this is the key difference. Spiking is a motif or an

architecture that is going to enable us to move from the batch-based setting to the real-time or the online setting. The neural agents that we are ultimately concerned with process sensory stimuli, which are sampled from an environment or a stochastic process over time. So here we see a really nice connection to epistemic foraging, information foraging in active inference. And here are some specifications of

what is meant by that, and also more about the data stream formulation. It still treats each data point as a discrete element, though spiking neural networks are intended to process input stimuli that arrive with a continuous

flow of time. In order to truly adapt the above stream to the spiking neural net, we further generalize the settings

so that each tuple is presented to the learner for a fixed period of simulated time. And the length of the stimulus and

interstimulus times will vary depending on the data problem and application. So it's not the case that it's just an infinitely short flash. It's the case that it's there for a defined amount of time, kind of like a video frame. So if a video is 25 or 30 frames per second, then that is in this sort of 30 to 50 milliseconds range of presentation of a given stimuli. So that allows the connection between the frequency of the regularities in the pattern in the environment to be connected to the rate of updating and of like the real biophysics of neuronal membranes or of some neuromorphic computer. 2.2 gives a mechanistic description of the spiking neural coding network. So this is the spiking based generalization. More formalisms and information. Here's what it looks like in Figure 1. A two-layer spiking neural coding network architecture.

We see a lot of motifs from the previous papers. This kind of local loop and the E's and the Z's. Green diamonds indicate the error units E, which in this simplified model depiction compute the amount of mismatch between the current predictions, which are the Z's of sub- μ , and the target signal traces Z's of T. Variables s represent the binary spike outputs of a particular vector grouping of neural units at time T. So more formalisms are provided. And just like the last paper, there is an algorithm in the appendix. Here's what the spNCN algorithm looks like.

Figure 2. I'm just overlaying Figure 1 just on top. It's not meant to overlay it directly, but just to remember that this is the kind of architecture that spike trains are being passed through or are occurring within. So Figure 2 is showing these kind of biologically reasonable, despite the simple integrate and fire model, traces. Kind of like a single neuron recording from inside of this little in silico brain. Section 3 in silico results with the model simulation. First, they looked at the MNIST, the digit classification data set. They presented two sets of experiments that tested the SPNCN's ability to generate and classify patterns, as well as its ability to retain knowledge over task sequence, and the appendix has more experiments. So Table 1, 2, and 3 have different evaluations and scores. And then also, just like the t-SNE that Blue mentioned in the first paper, here's the t-SNE now of the SPNCN with access to MNIST training up front. In the middle, the continual SPNCN, and a continual SPNCN with a buff versus the lat. So we'll go into more detail with these subvariants, but here we can see that this real-time learning is actually recapitulating similar structure and

near enough similar learning performance as batch learners, which is a big theme of the paper. So here is a figure 5, showing task accuracy normalized to 0, 1 over proportion of a current task data. And so on the x-axis is the portion of the task completed. On the y-axis is the accuracy range. So here it's shown for the three SPNCN variants, but that was done for not just one data set, but that was actually done for a whole set of data sets in figure 5. And so the big takeaway from this whole figure is that SPNCN obtains competitive cross-task performance even though it is trained online. It only processes data patterns once, whereas to obtain top performance, rate-coded baselines were trained on each task for multiple epoch. Pretty cool. And then figure 4, again, is a t-SNE layout of these different model variants. 3.2 describes some limitations. And then the conclusion. In this paper, we propose spiking neural coding, as well as a concrete model instantiation composed of leaky integrate and fire neurons and its learning process for adjusting synaptic weight strengths incrementally. Pattern classification and continual learning experiments demonstrate the generalization ability of our model in the realm of online continuous time-based learning. And the model is competitive with many spiking neural networks learned with spike time-dependent plasticity, STDP, or other mechanism. Further, our model exhibits better computational economy and is more biologically plausible than modern-day non-spiking artificial neural networks. And then it'll be cool to discuss future work. So to kind of close this paper and head into our fourth one, we started with a big picture with predictive coding, things that we've talked about in many live streams. We just heard in the last two papers, the convolutional extension, which enables higher resolution naturalistic images. And then this spiking variant, which brings it into the setting of real-time online learning through spiking, which also has lower energy requirements than non-spiking neural networks. And now, back to blue for the fourth paper, Cog and Gen building the kernel for a hyperdimensional predictive processing cognitive architecture. Cool. Thanks, Daniel. That was neat. Do you want to read this abstract? Sure. We present a new cognitive architecture that combines two neurobiologically plausible computational elements. One, a variant of predictive processing known as neural generative coding, NGC, and two, hyperdimensional slash vector symbolic models of human memory. We draw inspiration from well-known cognitive architectures such as ACT-R, SOAR, Liabra, and Spown-Nengo. Our cognitive architecture, the cognitive neural generative system, Cog and Gen, is in broad agreement with these architectures, but provides a level of detail between ACT-R's high-level symbolic description of human cognition, and Spown's low-level neurobiological description. Cog and Gen creates the groundwork for developing agents that learn continually from diverse tasks and model human performance at larger scales than what is possible with existent cognitive architectures. We aim to develop a cognitive architecture that has the power of modern machine learning techniques while retaining long-term memory, single-trial planning, transfer learning, planning, and other capacities associated with high-level cognition. We test Cog and Gen on a set of maze learning tasks, including mazes that test short-term memory and planning, and find that the synergy between its predictive processing and vector symbolic components allow it to master the maze tasks. Future work includes testing Cog and Gen on more tasks and exploring

[illegible]

Okay, so neural building blocks adding to memory. So this is this reference that I told you we were going to talk a lot about from 1984. I think it's some it's at 1986 somewhere. Maybe there's like an

updated model. But Minerva 2 stores memory traces, observations, or sequences of observations of the state of the world. Each memory trace is represented as an array of real numbers, a vector. Minerva 2 stores all memory traces separately as rows in a continuously growing table. We use Minerva 2's forgetting mechanism. Oh, that's the 1986 reference to randomly delete values from memory each time memory is updated at sufficiently high probability to impose a computationally tractable limit on the size of the memory. So if there was no forgetting, there would be like this would be an impossible problem, just an impossible method by which to store the memory. Okay, next.

So this is how you get things out of memory. So in Minerva 2, memory retrieval is not a lookup process. It's a reconstruction process. So when a retrieval queue or a probe is presented, like the smell of chocolate chip cookies, each vector in the memory table is activated in proportion to its similarity to its similarity to the queue. So similarity is computed as a normalized dot product of the queue's vector with the stored vector. So every stored vector is activated by its similarity to the probe raised to a power. And by raising the similarity to some power, the contribution of the most similar vectors is emphasized.

And then this is the what I was talking about at the very beginning about an echo. So it says here an echo is retrieved from memory as a weighted sum.

So that's this equation in the top right. So where ME is the echo or the output from memory. MI is the ith trace in memory.

MP is the probe or input to memory. And M is the number of traces in the memory table.

And so we're going to look in the next slide about so is is like prediction just an echo from memory. So we're going to look at that maybe here. Oh yeah, it's here.

So it says in Cog and Gen, we combine insights from several versions of Minerva 2. We make an architecture wide commitment to predict then correct learning.

Predict then correct can be implemented in Minerva 2 as discrepancy encoding. Given a sequence of observations, Minerva 2 can predict the next observation based on past experience.

After the prediction, a new observation is made. We update memory with the difference between observation MX and prediction ME. So it was not just an echo.

Is prediction an echo from memory? Like I think that that's what they're saying here. So MX minus ME. So observation minus prediction such that if the prediction is correct, no change is made to memory.

Whereas if the prediction is wrong, that prediction is inhibited in similar future contexts.

This is awesome. I love that you're pulling this kind of echo and prediction and memory because it's also related to how we

learn. Like we predict then correct. And that's kind of like the prepare, measure, learn, quantum information, quantum FEP connection.

But then how rich could a learning experience be if we predicted, what do you think the next slide is going to be?

Like then I learn about you. I learn about the paper and then no skin in the game. It comes up. I was off.

I was off because I didn't know you learn that's the update and then you learn on the learning.

So there's no end to how many layers you could learn with just like one data point.

Right. Like what's my prediction of the next slide? I theoretically have that in my memory as an echo somewhere, but it's not coming to mind.

So we'll find out when we get there.

All right, here we are.

So short term versus long term memory.

So for Cog and Jen, we adopt Collins et al's 2020 approach and model both working and declarative memory using Minerva 2.

Our working Minerva 2 is cleared after a task is completed.

For example, a maze is solved.

Whereas the contents of the declarative Minerva 2, which serves as the episodic memory in this paper's instantiation of Cog and Jen,

persist across tasks.

So there's some memory clear when the task is over.

Yeah. Short term memory that gets cleared after like each example, like each question on a test, each maze.

But then the declarative way of knowing is being kept episodically.

So like a memory of solving that maze is stored declaratively.

But each maze is each only being addressed with the working memory.

So that's cool.

So there's not a lot here in the perception place.

So they say that the environment that they investigate, which is Jim Mini Grid, which is an open AI environment.

So if you guys are interested, you can go check out open AI Jim Mini Grid.

But it provides a fixed problem specific encoder FE and decoder FG.

So they use a fixed encoding and decoding scheme to simplify the simulation.

And future work is going to investigate learning the perception modules as neural generative coding circuits instead.

So they didn't do that for this paper.

Next paper.

That's coming.

Okay.

Okay.

So neural generative procedural memory.

Action selection is driven by changes in the neural activity of basal ganglia, which estimate the value of the expected reward.

And these authors were motivated by the finding of expected value estimation in the brain.

So they made the cog and gens procedural module implement a neural circuit that produces intrinsic reward signals.

At a high level, this neural machinery facilitates some of the functionality offered by the basal ganglia and procedural memory, simulating an internal reward creation process.

So they say, we implement a neural generative coding dynamics model from which a reward signal is calculated as a function of its own error neurons.

We couple the dynamics model with a short-term memory module based on Minerva 2, which adjusts the reward value produced by the dynamic circuit by determining if an observed state is familiar or not.

Yeah.

Yeah.

Yeah.

Yeah.

And then...

Well, just, you're lacking...

You're going to see that in a figure, in the next figure.

Okay.

I'm just going to...

I'm going to let your internet just return.

Okay.

Okay, Blue?

I shut off my video.

Okay.

Okay.

Yeah.

The only comment on this is this idea of the intrinsic reward signal happening from a procedural module.

Yeah.

I'm good.

Can you hear me?

Yeah.

So it's like epistemic foraging.

You have a procedure for foraging.

And then there's an intrinsic reward even arising just by carrying out the procedure, which might be the predict and learn and reflect dot zero dot one dot two with an intrinsic smooth reward signal rather than kind of a needle in the haystack.

Yeah.

like being just in a continuous flow.

Is your internet back?

I think so. Can you hear me?

Okay. Yes. Figure three then continue.

Okay. Nice.

You can hear me?

Yep. Go for it.

Okay. Perfect.

I just, I turned off my video.

So if I get the flash again,

I'll turn it off because sometimes it's easier.

Okay. Figure three.

Perfect.

So, and just what you were saying

about the intrinsic reward

before I delve into figure three,

they did talk a little bit in this paper,

and I don't know if I pushed it into here,

but maybe we'll get into it in the dot two,

but they did talk about trying to push exploration

via an intrinsic reward signal.

So we can kind of get into that exploration exploitation

discussion in the dot two a little bit more.

Okay. So figure three, the Cog and Jen dynamics model.

It says that arrows transformed data

across similar across synapses.

And then the gray circles represent stateful neurons

and the green diamonds represent error neurons.

So this is the multi-layer model that we're looking at here

with an external input and Z of T .

And we can see the layers across time.

So I think it's maybe it says more in the next slide.

Let's see.

So building upon the notion of planning is inference.

We generalize neural generative coding to the case of action driven tasks,

which we call active neural generative coding.

Notably within the neural generative coding action motor model is a modifiable working memory

that allows the model to store a finite quantity of projected latent state vectors

into a set of self-recurrent memory vector slots.

This particular working memory module, which we call the self-recurrent slot buffer, serves as the glue that joins the models of Cog and Jen together.

So we talked, we heard about the slot buffer a long time ago.

And here we go back to the slot buffer.

And then in the next figure, we're going to look at the action driven task in the next figure.

Perfect.

So this is the motor model where the internal control portion produces signals.

Those are the CINT that manipulate working memory and the external control portion transmits signals.

So this is the CXT down at the bottom that affects the environment.

And so both the internal and external control share the same working memory, which is interesting.

So explicit declarative memory, episodic memory.

So Cog and Jen implements a replay mechanism in the form of an episodic memory constructed with Minerva.

Information is transferred to memory through an intermediate working buffer where pieces of the transition are progressively stored as they are encountered through the agent environment interaction process.

We do not update the motor action and dynamics models online, but instead update their parameters only when episodic memory is sampled.

Thus, the Cog and Jen computational process has two phases at each time step of the simulation, the processing step and the learning step.

And so we'll see figure two.

I think I have it next.

Yeah, perfect.

I reordered them because they were mentioned in different orders in the paper.

So Daniel showed this to us earlier.

Do you want to maybe talk a little bit about this figure while I go grab my computer plug?

Yeah.

Do you mind?

Cool.

Well, a few ways to look at it.

One of them is we're dealing with this fundamental interface question, blanket question, boundary question, which is how we begin in active inference and free energy principle about the agent in their boundedness, interfacing with the environment.

And then while we might see this in a partially observable Markov decision process generative model, here we basically see the formalism from the last several papers, this paper specifically, being represented in the way that we've seen agent environment interface before.

Observation coming in at time step t , latent space here.

It's actually a more sophisticated latent space.

It's not just like one Gaussian parameter that's getting updated.

There's this entire dialectic with the procedural and the working memory and the episodic, all of these cognitive features.

And then there is motor or action control.

Action is emitted within the time step.

And then there's the observation at the next time step.

So this is a cognitive architecture that brings together a lot of the features that we've been talking about, plus these higher order structured, purely cognitive aspects.

Thank you so much.

My computer thinks you as well.

So this, um, so I'm curious here in this model, how things get from the working memory into the episodic memory.

So I would like to see like maybe more details about that as we go into the dot two.

Um, yeah.

Okay.

So in the mini grid world problem, um, they are investigating two tasks, uh, to evaluate the agents that they used.

So one is the door key task, the DK task, and the other is the memory task.

So everybody's in a grid.

It's a partially observable view of the agent's environment.

Um, and there's a three channel object.

Um, yeah, so the agent's environment is a three channel object, a tensor created by mapping each visible grid cell to three integer values.

So everything, each tile contains either nothing, one object and a discrete object type.

So ultimately each file is encoded to an object index.

So it's unseen empty wall and then a color index, red, green, whatever, and then a state index, which is open, closed or locked.

So those are the three, um, three.

Yeah.

Things, three things, three things in the environment.

Okay.

Next.

So.

The agent is restricted to picking up one single object, a key and may open a locked door if it carries a key that matches the door color.

So that's the door key problem.

The discrete action space for the agent is six actions, turn left, right, forward, pick up an object or drop the object that it's currently carrying or activate.

Such as like opening a door or interacting with an object.

The reward signal provided by all tasks is sparse.

The agent is only given a positive extrinsic reward if it, extrinsic reward, if it reaches a green goal tile and otherwise making all problems difficult from a reinforce, reinforcement learning perspective because it's sparse.

And that's, that's like, again, at the very beginning, like I act, I asked about the sparsity and what is the, the, like, I mean, there's some overarching like level of complexity when there's a sparse set of rewards, a sparse set of problems, a sparse set of activated neurons, I mean, et cetera, et cetera, et cetera, right?

Sparse networks are hard to deal with mathematically and computationally.

And I guess in terms of programming a system with sparse learning, it's beneficial.

It seems that way anyway.

Okay.

So each problem has a specific time step, time step limit limit allotted to allow the agent to complete the task.

So you do it within a certain time where you fail.

Yeah.

Very interesting.

Their sparsity in architecture, space, time, reward, utility, epistemic value, all like we don't live in flatlands.

We live on the rough and variable peaks and the very embodied constraints and limitations of how things can actually be connected to each other.

So that's wild.

Cool.

Okay.

So for the door key problem, agent must find the key, the position of the key changes inside of a locker room and then pick up the key in order to unlock the door so it can leave the room. Upon entering the second room, the agent must find the green tile in order to successfully exit, terminating the episode by reaching the goal state to receive a positive reward.

So they use a six by six grid.

The memory task is a different test.

It's posed as a memory test.

So in the memory test, the agent starts in a small room where it can see an object, a key or a ball.

And they, they modified the problem to ensure that the agent starts looking in the direction of the object.

That's interesting.

I wonder why, like, so that they don't have to, it does just save time.

Like, so you don't have to look right, look front, look left, look behind you.

I wonder why they modified the problem in that way.

So after perceiving the object, the agent has to turn around, exit the room and go through a narrow hollow that ends in a split.

At the end of the split, the agent can either go up or down.

At the end of each of these splits is a different object, either a key or a ball.

To successfully complete the episode and receive a positive reward, the agent must remember the initial object that it saw and go to the split that contains the correct matching object.

So did you see a key or did you see a ball?

And then which way are you going to go at the end of the maze?

Um, and then they did a seven by seven grid for that one.

Okay, let's see what they found.

Oh, not yet.

So these are the models of comparison.

So they use their model.

And a standard DQ network model distillation, and that's the DQN, R and D, which is a curiosity model, and then a DQN that learns through account based formulation of the be bold exploration framework, which we'll have to learn more about in the dot two.

So these are the models that they tested against.

And then this is what they found.

So success rate in the memory task for the cog and gen was like off the hook, like blew the other ones out of the water, right?

Um, like 98 success rate for the memory task, which is pretty crazy.

Um, and then the door key task was, I guess like zero for the DQN and then a hundred percent, a hundred percent, a hundred percent for the other ones.

Um, or maybe then the reverse.

So let's see from left to right, the door key task, the memory task in the bottom row is success rate over the last hundred episodes.

The right is the episode length.

Oh, there you go.

The percentage in case of the maximum worst case episode length.

So that is, um, DQN having the worst, right?

And then cog and gen wasn't the fastest.

So, oh yeah, you can see the episode.

Yeah.

That's crazy.

I think it's just a pure coincidence that this one's a hundred point zero, but this is a hundred percent zero, like a hundred percent.

And this one's 100 time steps point zero, but either way, DQN does worse than all three, which can solve the problems, but a problem, but the cog and gen is twice as good on the mem problem.

And then the episode lengths are pretty comparable.

And except the DQN.

Yeah.

Yeah.

This one's just kind of a baseline.

It takes forever.

All right.

It takes forever and doesn't do very well in the memory task.

Um, so yeah, there you go.

All right.

So that's how, that's a lot of time on the model and very little time on the results.

Uh, and then this is just some more of the results here.

So, um, you can just see the, the average reward episode length, um, smooth for the door key problem and the memory task, which is the right.

So the door key is the left.

The memory task is the right.

Um, and here you can see the cog and gen on this bottom one.

Do you see this curve?

That is the, um, the curve depicts the mean and standard deviation over five trials.

And the left is the average reward, I think.

Oh yeah.

Episodic reward.

So that is the, where we saw 98 for the cog engine.

Um, and that's here too for the door key task memory task.

Okay.

Results and discussion based on the results.

We find that cog and gen is able to learn the grid world task.

The performance is comparable to, or on par with powerful deep intrinsic curiosity.

Reinforce reinforcement learning methods that have access to problem specific global information.

And cog engine can successfully solve and outperform all baselines on the memory task.

So bravo, we have arrived.

Wow.

Blue.

What a ride.

Yeah.

What do you think about the four paper, the four paper, uh, model?

Well, I can only return to what Alex wrote in an email, which is, it makes sense to have done it, how it was done.

I think it'd be easy to look at a cognitive architecture like this, which integrates the analytical components and also these more psychological or cognitive features like epistatic and procedural memory.

It's easy to look at that in a way that's disconnected from the actual computational implementation because connecting up the formalism feels as easy as just writing an equation or just drawing something.

And then to say, well, maybe, maybe the working memory, you know, sends a message to this other buffer.

It's like, you can just draw that out and nothing's going to stop you.

But then the first three papers kept it connected very closely to the actual locally implementable computation.

So then we were able to get to this point of, um, looking at such an interesting cognitive model that's also performant and be able to talk about where it could be applied or what would be different or limitations or like these different features of this.

But it, it feels very different to look at this after the first three, which is the research direction.

So I think it was a great selection and I'm just looking forward to talking with Alex.

What about you?

Yeah, I'm looking forward to like diving deeper into, um, the, the models and kind of understanding more about like getting under the hood, maybe so to speak.

So what, what makes them tick in a little bit more detail?

So should be good.

Cool.

Any other comments?

Awesome.

Well, thank you for preparing and having fun working on this dot zero leading up to now.

And yeah, in the coming weeks, we will proceed with 56.1 and 0.2.

So thank you everybody.

Until next time.

Yeah, thank you.

Thank you.